

29/9/23

Assignment-2

Tutorial sheet-4

Section-A

Q1 why stack is called a LIFO data structure?

Ans stack is called a LIFO (Last-In-First-Out) data structure because the last element pushed onto the stack is the first one to be popped off. It follows a "last in, first out" order.

Q2 the time complexity of converting a prefix notation to infix notation is

Ans the time complexity of converting a prefix notation to infix notation depends on the length of the expression but is generally considered to be $O(n)$, where 'n' is the length of the expression.

Q3. what would be the solution to the given prefix notation?
- + 5 / 10 5 5

Ans	Symbol	Stack	My intermediate result
	5	5	
	5	5, 5	
	10	5, 5, 10	
	/	5, 2	$10/5 = 2$
	5	5, 2, 5	
	+	5, 7	$5 + 2 = 7$
	-	2	$7 - 5 = 2$

The solution to given prefix notation is 2.

Q4 state the different ways of representing expressions.

Ans Different ways of representing expressions are:

- (i) Infix Notation - In this operators are placed in between the operands. eg - $a+b$, $\langle \text{operand} \rangle \langle \text{operator} \rangle \langle \text{operand} \rangle$
- (ii) Postfix Notation - In this operators are placed after the operand. eg $ab+$, $\langle \text{operand} \rangle \langle \text{operand} \rangle \langle \text{operator} \rangle$
- (iii) Prefix Notation - Here operator is written first followed by an operand. $\langle \text{operator} \rangle \langle \text{operand} \rangle \langle \text{operand} \rangle$
eg. $+ab$.

Q5. What is the number of moves required in the tower of Hanoi problem for k disks?

Ans $2^k - 1$ is the number of moves required in the Tower of Hanoi problem for k disks.

Section - B

Q1 Derive the equivalent infix expression and value for the postfix form $1\ 2\ +\ 3\ *\ 4\ 5\ *$

Ans Equivalent ~~infix~~^{infix} expression is $(1+2)*3-(4*5)$

Symbol	Stack	Intermediate result
1	1	
2	1, 2	
+	3	$1+2 = 3$
3	3, 3	
*	9	$3*3 = 9$

4 9, 4

5 9, 4, 5

* 9, 20

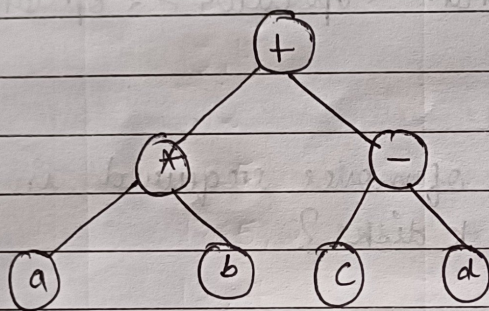
- -11

$$4 * 5 = 20$$

$$9 - 20 = -11$$

The value for the postfix expression is -11.

Q2 From the given expression tree. Find out the postfix expression



Ans Infix expression : ~~(a+b)*~~ $a * b + c - d$

~~ab* + c - d~~

postfix expression :

~~ab* c~~

Symbol	Stack	expression
a	(	a
*	(*	a
b	(*	ab
+	(+	a*b ab*
c	(+	a*b c ab*c
-	(-	a*b c + ab*c +
d	(-	a*b c + d ab*c + d
)		a*b c + d - ab*c + d -

The postfix expression is $ab * c + d -$

3. What is enqueue dequeue in stack? explain with example.

Ans enqueue means to insert an item into the back of the queue.

Dequeue means to remove the front item.

Example - Suppose you have a queue of people waiting in line at a ticket counter. When a new person arrives and joins the line, they are enqueued at the rear of the queue.

Dequeue - This operation removes an element from the front or "head" of the queue. It's similar to the person at the front of the line getting their ticket and leaving.

Example - In the same ticket counter scenario, when the ticket agent serves the person at the front of the queue, the person is dequeued, and the next person in line becomes the new front of the queue.

Section - C

Q1 What do you understand by stack overflow and underflow? Describe the evaluation of postfix expression with an example. What is the time complexity of enqueue and dequeue in stack.

Ans A stack overflow occurs when you attempt to push an element onto a stack that is already at its maximum capacity.

A stack underflow occurs when you attempt to pop an element

from the stack that is already empty.

evaluation of postfix expression involves scanning it from left to right and using a stack to keep track of operands.

eg: $3\ 4\ *\ 5\ 2\ +\ -$

Symbol	stack	intermediate result
3	3	
4	3, 4	
*	12	$3 * 4 = 12$
5	12, 5	
2	12, 5, 2	
+	12, 7	$5 + 2 = 7$
-	5	$12 - 7 = 5$

Time complexity of enqueue and dequeue in stack typically known as push or pop is $O(1)$ as these operations only update the TOS and do not depend on the size of the stack.

Q2. How insertion and deletion works in stack? If the sequence of operations - push(1), push(2), pop, push(1), push(2), pop, pop, push(2), pop are performed on a stack, what will be the sequence of popped out values.

Ans

push(1)	→	[1]
push(2)	→	[1, 2]
pop	→	[1]
push(1)	→	[1, 1]

push[2] $\rightarrow$ [1, 1, 2]
 pop $\rightarrow$ [1, 1]
 pop $\rightarrow$ [1]
 pop $\rightarrow$ []
 push(2) $\rightarrow$ [2]
 pop $\rightarrow$ [2]

Sequence of the popped out values: 2, 2, 1, 1, 2.

Q.3(i) The following postfix expression with single digit operands is evaluated using a stack. Find output
 $7\ 3\ 4\ ^\wedge\ /3\ 4\ * + 6\ 2\ * -$

(ii) convert infix expression into postfix expression using stack $(P+Q)*(R*S-T)*U/V$ is?

Ans (i) symbol	stack	Intermediate expression
7	7	
3	7, 3	
4	7, 3, 4	
$^\wedge$	7, 81	$3^4 = 81$
/	7 81	7 81
3	7 81, 3	
4	7 81, 3, 4	
*	7 81, 12	$3 * 4 = 12$
+	7 81 + 12	
6	7 81 + 12, 6	
2	7 81 + 12, 6, 2	
*	7 81 + 12, 12	$6 * 2 = 12$
-	7 81	$7 81 + 12 - 2 = 7 81$

ii) Symbol	Stack	Expression
(	(	
(	((	
P	((	P
+	((+	P
Q	((+	PQ
)	(	PQ+
*	(*	PQ+
(	(*	PQ+
R	(*	PQ+R
*	(*	PQ+R
S	(*	PQ+RS
-	(*	PQ+RS*
T	(*	PQ+RS*T
)	(*	PQ+RS*T-
*	(*	PQ+RS*T-*
U	(*	PQ+RS*T-*U
/	(/	PQ+RS*T-*U*
V	(/	PQ+RS*T-*U*V
)		PQ+RS*T-*U*V/

The postfix expression is $PQ+RS*T-*U*V/$